Neural Predictive Control to Coordinate Discrete- and Continuous-Time Models for Time-Series Analysis with Control-Theoretical Improvements

Haoran Li
lhaoran@asu.edu
Arizona State University
Tempe, Arizona, USA

Muhao Guo
mguo26@asu.edu
Arizona State University
Tempe, Arizona, USA

Yang Weng
yweng2@asu.edu
Arizona State University
Tempe, Arizona, USA

Hanghang Tong
htong@illinois.edu
University of Illinois at Urbana-Champaign
Champaign, Illinois, USA

Abstract

Deep sequence models have achieved notable success in time-series analysis, such as interpolation and forecasting. Recent advances move beyond discrete-time architectures like Recurrent Neural Networks (RNNs) toward continuous-time formulations such as the family of Neural Ordinary Differential Equations (Neural ODEs). Generally, they have shown that capturing the underlying dynamics is beneficial for generic tasks like interpolation, extrapolation, and classification. However, existing methods approximate the dynamics using unconstrained neural networks, which struggle to adapt reliably under distributional shifts. In this paper, we recast time-series problems as the continuous ODE-based optimal control problem. Rather than learning dynamics solely from data, we optimize control actions that steer ODE trajectories toward task objectives, bringing control-theoretical performance guarantees. To achieve this goal, we need to (1) design the appropriate control actions and (2) apply effective optimal control algorithms. As the actions should contain rich context information, we propose to employ the discrete-time model to process past sequences and generate actions, leading to a coordinate model to extract long-term temporal features to modulate short-term continuous dynamics. During training, we apply model predictive control to plan multi-step future trajectories, minimize a task-specific cost, and greedily select the optimal current action. We show that, under mild assumptions, this multi-horizon optimization leads to exponential convergence to infinite-horizon solutions, indicating that the coordinate model can gain robust and generalizable performance. Extensive experiments on diverse time-series datasets validate our method's superior generalization and adaptability compared to state-of-the-art baselines.

CCS Concepts

• **Computing methodologies** → **Learning latent representations**.



This work is licensed under a Creative Commons Attribution 4.0 International License.
KDD '26, Jeju Island, Republic of Korea
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2258-5/2026/08
https://doi.org/10.1145/3770854.3780261

Keywords

Time-series Analysis; Deep Learning; Latent ODEs; Model Predictive Control; Performance Guarantees

ACM Reference Format:
Haoran Li, Muhao Guo, Yang Weng, and Hanghang Tong. 2026. Neural Predictive Control to Coordinate Discrete- and Continuous-Time Models for Time-Series Analysis with Control-Theoretical Improvements . In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '26), August 09–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3770854.3780261

1 Introduction

Time-series analysis, such as classification, interpolation, and forecasting, has extensive applications in various domains like energy [33, 34], marketing [54], transportation [46], etc. Recently, introducing dynamical modeling to this analysis has been widely recognized. For example, the continuous feature flow brings interpolative capacities at an arbitrary time, causing substantial improvements for scenarios like irregularly-sampled time-series data [29, 51]. Additionally, the learned dynamics can naturally extrapolate future states, which is often more accurate than discrete updates in many smooth and continuous systems [3, 36] or even in some stochastic systems like the PhysioNet dataset [51, 58].

However, gaining accurate feature dynamics is challenging. There may exist a smooth dynamical function for many continuous systems, including engineering systems (e.g., power, gas, water, mechanical, and transportation systems) [37], chemical reactions [40], biological networks [47], weather systems [31], etc. However, for other time-series datasets, frequent disturbances or events could intermittently change dynamical evolution [35], which calls for a highly adaptive dynamical function to different contexts. As contextual information can be extracted from long-term historical patterns, it's critical to merge both the adaptive short-term dynamics and long-term patterns innovatively.

To understand temporal patterns, discrete-time Deep Learning (DL) methods have been widely utilized, such as Recurrent Neural Networks (RNNs) [17] or Long Short-Term Memory (LSTM) [26] and Transformer-based models [63, 66]. They process discrete observations sequentially but ignore the intra-dynamics between adjacent observations. Consequently, many studies demonstrate

the model's insufficiency to tackle data from continuous systems [16, 29, 43, 51, 56]. Moreover, without dynamical knowledge, the performance of discrete-time DL degenerates significantly if the data is irregularly sampled [51].

To incorporate dynamics, existing work can be categorized into the following groups. First, the dynamics between observations can be explicitly learned via methods such as Neural Ordinary Differential Equation (Neural ODE) [14]. However, Neural ODE depends on the initial value without adaptation capacities [29]. The dynamics keep changing for complicated time series with disturbances and events. There needs to have a mechanism to adjust the dynamical evolution. Therefore, the second group increases the capacity of the model by adding control actions to the data dynamics. For example, Neural Controlled Differential Equation (Neural CDE) [29] and Neural Rough Differential Equation (Neural RDE) [43] create a rough path based on observed data to control the evolution of the dynamics. Unsupervised DL with control theory is also proposed in [21]. Another popular model is a learnable State Space Model (SSM) [2, 23, 24, 56, 59]. SSM-based methods linearly combine feature states and control vectors. Generally speaking, these continuous-time models rely on observational data as control actions and overlook the high-level temporal patterns from the past [16]. The historical information of time series, however, is critical to providing contextual information and controlling dynamical propagation.

There has been limited exploration into integrating discrete- and continuous-time models for time-series tasks. ODE-RNN [51] and GRU-ODE [18] use discrete-time hidden states to modulate continuous dynamics, leveraging RNN architectures to inform ODE evolution. [16, 27] combine Neural ODEs or Neural CDEs with Transformer architectures [60], using attention mechanisms to compute control actions. However, from a control-theoretic perspective, these approaches are inherently non-robust. They primarily focus on *single-horizon* evaluations by optimizing the current action for immediate objectives. This neglects the long-term influence of actions on future feature trajectories. As a result, the induced dynamics may not converge to a stable equilibrium or a desired reference trajectory [30].

In this paper, we cast the Neural ODE-based time-series learning problem as an optimal control problem [42, 52]. Hence, we show that the model's performance and adaptation capacity are deeply linked to the convergence of the control problem. For example, (1) **Time-series classification**. The stable and high-level feature that corresponds to a label can be viewed as an equilibrium point of the feature ODE flow. The perturbations or distributional shift of the time series demand control strategies that always stabilize the ODE flow to the equilibrium. (2) **Time-series interpolation/extrapolation**. The reference feature flow can correspond to the true time-series dynamics for highly accurate interpolation and extrapolation. Effective control actions can enable converged approximation to these reference trajectories. Therefore, we propose a control-theoretical approach to design efficient *actions* and *control-based training algorithms*, providing certain convergence guarantees.

Instead of utilizing current data or features as the control action, such as those in ODE-RNN or Neural CDE, we propose to extract the temporal features and forecast a sequence of actions for the future.

Table 1: Table of Notation

Scalars	
t_i	Time for the ith observation
N	Total number of observations for the time-series
M	Number of horizons for the optimization in MPC
y	Label of the time-series
λ	Regularization penalty coefficient
Vectors	
$\boldsymbol{x}_i$	The ith observation of the time-series
$\boldsymbol{z}_i$	The ith discrete hidden state in an RNN
$\boldsymbol{h}(t_i)$	Continuous hidden state evaluated at time t_i
$\boldsymbol{u}_i$ or $\boldsymbol{u}(t_i)$	Action vector to control $\boldsymbol{h}(t)$ flow at time t_i
Matrices	
$U_{i,M}$	M-horizon control sequence
$H_{i,M}$	M-horizon sequence of $\boldsymbol{h}(t)$
Functions	
$g_\psi(\cdot)$	Discrete-time updating function
$\ell^1_\psi(\cdot)$	Neural network encoding discrete states to actions
$\ell^2_\psi(\cdot)$	Neural network to map actions to label or data
$f_\phi(\cdot)$	Neural network for the derivative $\frac{d\boldsymbol{h}(t)}{dt}$
$\ell^1_\phi(\cdot)$	Neural network encoding $\boldsymbol{x}_1$ to $\boldsymbol{h}(t_1)$
$\ell^2_\phi(\cdot)$	Neural network to map state to label or data
$X(t)$	Interpolated continuous input path
$J(\cdot)$	Task-dependent cost function
$\hat{J}(\cdot)$	Regularization term for the actions
ODESolve$(\cdot)$	Function to solve the initial value problem
$U_i(t)$	Interpolated continuous control path from t_i to t_{i+M}

Specifically, we employ an auto-regressive discrete-time model, e.g., an RNN, to understand the temporal information and forecast future actions. The auto-regression aims to provide rich contextual information for the current and future horizons. Hence, the output actions are highly expressive representations that can effectively adjust the continuous-time model, e.g., a Neural-ODE-based model. In general, the overall framework is a coordinate model, where the discrete auto-regressor generates sequential action features to control the evolution of the ODEs in the continuous-time model. The new architecture not only becomes highly capable of extracting and fusing long-term temporal features and short-term dynamics but also embraces control-theoretical training and analysis.

Specifically, we conduct training by solving a *multi-horizon optimal control* problem, which chases convergence to the infinite-horizon solutions. A simple yet efficient approach is the so-called Model Predictive Control (MPC) [22, 30]. At each time, MPC solves a multi-horizon optimization to maximize the predictive performance and yield a sequence of optimal actions. Then, only the first control action is implemented. Such a process is repeated, bringing

rigorous convergence guarantees [61]. Mathematically, in Theorems 1 and 2, we introduce the *exponential convergence* to a stable equilibrium or a reference trajectory.

The overall model, dubbed Neural Predictive Control (NPC), leverages MPC during parameter updates. Specifically, with the produced control actions and the ODE dynamics, we construct a multi-horizon cost that can be iteratively minimized through gradient-based methods. In general, our NPC provides a unified framework for coordinating arbitrary pairs of discrete- and continuous-time models. It consistently outperforms each individual component, offers strong theoretical guarantees, and remains simple to implement. We validate its effectiveness on both synthetic and real-world time-series datasets, demonstrating substantial gains: we observe a 5% $\sim$ 15% improvement in classification accuracy and a 30% $\sim$ 60% reduction in mean squared error for regression tasks. Moreover, our framework remains highly scalable for high-volume and multi-dimensional time series, thanks to the highly efficient parallel computations in the discrete- [17] and continuous-time models [14].

2 Problem Formulation

In this paper, we aim to solve the following problem.

- Goal: Build an accurate time-series classifier or regressor by incorporating continuous-time dynamics.
- Given: A series of observations $\{\boldsymbol{x}_i\}_{i=1}^N$ at times $\{t_i\}_{i=1}^N$ with potentially *irregular intervals*. For a classification problem, the label y of these observations is also available.
- Find: A well-trained classifier or regressor to identify time-series labels or conduct accurate time-series interpolation or extrapolation.

Then, we introduce some preliminaries.

Discrete-Time Model. We consider the sequence DL, such as RNNs, that contains a hidden state $\boldsymbol{z}$ to store temporal information. $\boldsymbol{z}$ can be updated at discrete time whenever a new observation is input to the model. Specifically, at time t_i, the updating function is:

$$\boldsymbol{z}_i = g_\psi(\boldsymbol{z}_{i-1}, \boldsymbol{x}_i), \tag{1}$$

where $g_\psi(\cdot)$ is a neural network with a parameter set ψ, e.g., a cell block in RNNs [17]. The updated hidden state can be converted to the time-series label or forecast observations through an output layer. In our NPC framework in Section 3, we will convert the state to a sequence of actions, leading to high-level temporal representations to capture current and future contextual information to adjust the continuous dynamics.

Remark: There could be other candidates for the discrete-time model, such as Transformer-based models [66]. In essence, a model is qualified as long as it can extract temporal information and produce sequential action representations.

Continuous-Time Model. There are extensive Neural ODE variants that can model continuous-time dynamics. Their core component is the continuous feature ODE flow. Specifically, let $\boldsymbol{h}(t)$ denote the continuous feature states. A neural network $f_\phi(\cdot)$ is employed to depict the derivative of $\boldsymbol{h}(t)$:

$$\frac{d\boldsymbol{h}(t)}{dt} = f_\phi(\boldsymbol{h}(t), t), \tag{2}$$

where ϕ is the parameter set of $f_\phi(\cdot)$. The derivative in Equation (2) can be adapted with respect to input time t. However, such a capacity is limited. Recent advances promote the adaptation capacity through introducing an additional context vector $\boldsymbol{u}(t)$.

$$\frac{d\boldsymbol{h}(t)}{dt} = f_\phi(\boldsymbol{h}(t), \boldsymbol{u}(t), t), \tag{3}$$

For example, in Neural CDE [29], $\boldsymbol{u}(t) = \frac{dX}{dt}(t)$, where $X(t)$ is a continuous path created by the interpolation of the observations $\{\boldsymbol{x}_i\}_{i=1}^N$. Specifically, the interpolation uses a natural cubic spline with knots at $\{t_i\}_{i=1}^N$ such that $X(t_i) = (\boldsymbol{x}_i, t_i)$. In Augmented Neural ODE [19], $\boldsymbol{u}(t)$ is an auxiliary augmented vector.

Hybrid Model. In a hybrid framework, a discrete-time model $g_\psi(\cdot)$ is utilized to extract past information and provide more sophisticated context vectors. For example, in ODE-RNN [51], $\forall t_i \le t \le t_{i+1}$, we have:

$$\frac{d\boldsymbol{h}(t)}{dt} = f_\phi\big(\boldsymbol{h}(t), g_\psi(\boldsymbol{z}_{i-1}, \boldsymbol{x}_i), t\big), \tag{4}$$

In this formula, the context vector is defined as $\boldsymbol{u}_i = \boldsymbol{u}(t_i) := g_\psi(\boldsymbol{z}_{i-1}, \boldsymbol{x}_i)$. Basically, the discrete state $\boldsymbol{z}_i$ in the RNN (see Equation (1)) works as the context vector to control $\boldsymbol{h}(t)$ flow from t_i to t_{i+1}. Unlike the continuous control signal $\boldsymbol{u}(t)$ in Equation (3), the hybrid model employs a discrete context vector to improve efficiency. This design is justified, as the context vector can effectively capture local and relatively invariant environmental information. In Section 3, we show that our framework can employ both discrete and continuous control patterns.

Train the Hybrid Model by Solving the Optimal Control. During training, $\boldsymbol{u}_i$ is also optimized through updating the parameter set ψ of the RNN. Hence, we can view $\boldsymbol{u}_i$ as the control action, the RNN as the controller, and the learning problem as an optimal control problem. Specifically, we have:

$$\begin{aligned} \arg\min_{\psi,\phi} \ & J(\boldsymbol{h}(t)), \\ \text{subject to } & \boldsymbol{h}(t_1) = \ell_\phi^1(\boldsymbol{x}_1), \\ & \boldsymbol{u}_i = g_\psi(\boldsymbol{z}_{i-1}, \boldsymbol{x}_i), \\ & \boldsymbol{h}(t_{i+1}) = \text{ODESolve}\Big(f_\phi\big(\boldsymbol{h}(t_i), \boldsymbol{u}_i, t\big), \boldsymbol{h}(t_i), [t_i, t_{i+1}]\Big) \end{aligned} \tag{5}$$

where $J(\cdot)$ is a cost function evaluated at the hidden trajectory of $\boldsymbol{h}(t)$. $J(\cdot)$ is task-dependent and can be written by the cross-entropy (CE) loss in the classification or the mean square error (MSE) in the regression, which will be fully explained in Section 3. ODESolve$(\cdot)$, defined in [14], solves the initial value problem in $[t_i, t_{i+1}]$, given the initial state $\boldsymbol{h}(t_i)$ and the derivative $f_\phi\big(\boldsymbol{h}(t_i), \boldsymbol{u}_i, t\big)$. $\ell_\phi^1(\cdot)$ is a neural network to encode the initial data $\boldsymbol{x}_1$ to an initial continuous hidden state.

However, the control is single-horizon. As shown in the equality constraint, the action $\boldsymbol{u}_i$ at t_i will determine the flow of $\boldsymbol{h}(t)$ from t_i to t_{i+1}. Nevertheless, the optimal action, under some optimal parameters ψ^*, can't ensure the robust performance for the subsequent flow after t_i. The problem is heavily discussed in dynamic programming, Reinforcement Learning (RL), and MPC [9]. In this paper, we adopt MPC theory to renovate the training procedure

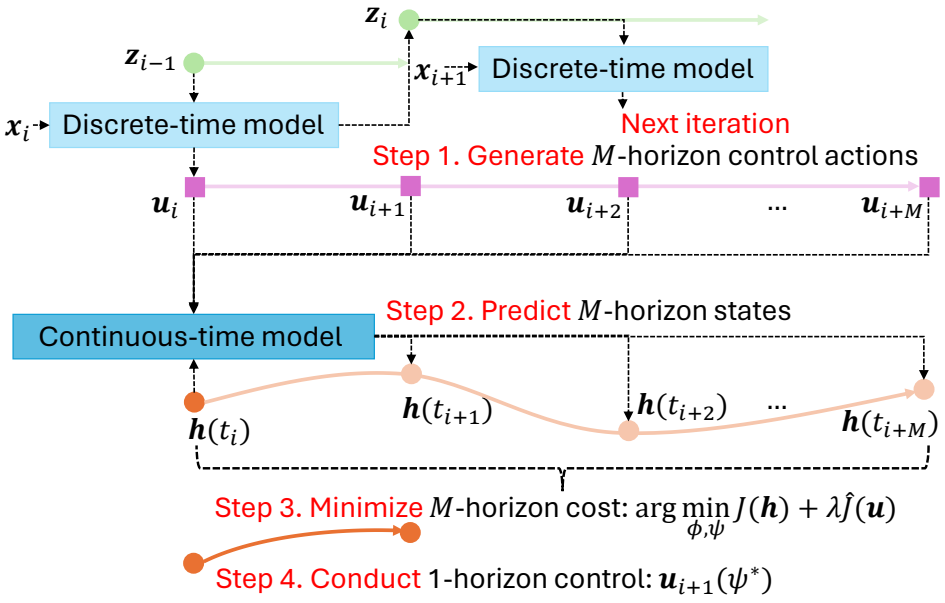


Figure 1: The framework of the proposed NPC. Steps 1-4 demonstrate how to conduct M-horizon training at t_i.

in Equation (5) since MPC is suitable and practical for the optimal control problem. As a result, innovative model architecture and training algorithms are proposed, which completely changes the paradigms from existing Neural ODE-based time-series analysis.

3 Methods

In this paper, we propose a highly expressive and theoretically grounded model, Neural Predictive Control (NPC). The overall framework is shown in Fig. 1. Generally speaking, NPC assigns a discrete-time model (light blue box) to generate M-horizon future actions $[\boldsymbol{u}_i, \cdots, \boldsymbol{u}_{i+M}]$ (pink squares). These actions are input to a continuous-time model (dark blue box) to modify the evolution of a continuous hidden feature state $\boldsymbol{h}(t)$ (light brown curve). Then, following MPC's philosophy, an M-horizon optimization is solved and a 1-horizon optimal action is conducted (dark brown curve). This approximates the solution of the infinite-horizon problem with a bounded and affordable error (see Theorems 1 and 2). Steps $1 \sim 4$ in Fig. 1 summarize the process and should be repeated sequentially. Specifically, we elaborate on the procedure as follows.

Discrete-time model to output predictive control sequence. As shown in the top layer of Fig. 1, we utilize the following discrete-time model:

$$\boldsymbol{z}_0 = \boldsymbol{0}, \boldsymbol{z}_i = g_\psi(\boldsymbol{z}_{i-1}, \boldsymbol{x}_i), [\boldsymbol{u}_i, \boldsymbol{u}_{i+1}, \cdots, \boldsymbol{u}_{i+M}] = \ell_\psi^1(\boldsymbol{z}_i), \tag{6}$$

where $\ell_\psi^1(\cdot)$ encodes the discrete state to M-horizon control sequence $U_{i,M} := [\boldsymbol{u}_i, \boldsymbol{u}_{i+1}, \cdots, \boldsymbol{u}_{i+M}]$. These predictive actions determine the propagation of hidden states $\boldsymbol{h}(t)$ in the continuous-time model.

The Proposed Predictive Optimal Control Framework. Based on the predictive action sequence, we propose a new optimal control paradigm. Mathematically, let $H_{i,M} := [\boldsymbol{h}(t_{i+1}), \cdots, \boldsymbol{h}(t_{i+M})]$, we introduce the following iterative M-horizon subproblem:

$$\begin{aligned} \arg\min_{\psi,\phi} \ & J(H_{i,M}) + \lambda\hat{J}(U_{i,M}), \\ \text{subject to } & \boldsymbol{h}(t_1) = \ell_\phi^1(\boldsymbol{x}_1), \\ & U_{i,M} = \ell_\psi^1(\boldsymbol{z}_i), \\ & H_{i,M} = \text{ODESolve}\Big(f_\phi(\boldsymbol{h}(t_i), U_{i,M}, t), \boldsymbol{h}(t_i), [t_i, \cdots, t_{i+M}]\Big) \end{aligned} \tag{7}$$

Compared to Equation (5), important changes, induced by the predictive control strategy, are marked in blue. $\lambda > 0$ is a penalty constant and $\hat{J}(\cdot)$ is a regularization term for the actions. The detailed formula for $J(\cdot)$ and $\hat{J}(\cdot)$ are presented in Equations (10) to (12). In this optimization, starting from an initial state $\boldsymbol{h}(t_1)$, the action sequence controls the dynamics in the subsequent M horizons, causing the feature sequence $H_{i,M}$ that are evaluated and optimized. The evolution computation differs according to different continuous ODE models. In this paper, we investigate two popular models: Neural CDE and ODE-RNN, which serve as excellent representatives of continuous-time and hybrid models, respectively.

Predictive-sequence controlled Neural CDE. The original Neural CDE [29] is controlled by the raw data path $X(t)$. We generalize this idea and consider a *control path* $U_i(t)$ $(1 \le i \le N)$ of $M+1$ horizons, where $U_i(t_{i+k}) = (\boldsymbol{u}_{i+k}, t_{i+k})$ is formulated based on $\{\boldsymbol{u}_{i+k}, t_{i+k}\}_{k=0}^M$ obtained in Equation (6). Consequently, for $t_i \le t \le t_{i+M}$, our *predictive Neural CDE* updates the continuous state $\boldsymbol{h}(t)$ with Riemann–Stieltjes integral [45]:

$$\boldsymbol{h}(t) = \boldsymbol{h}(t_i) + \int_{t_i}^{t} f_\phi'(\boldsymbol{h}(s), s)\frac{dU_i}{ds}(s)ds, \tag{8}$$

where the dynamics $f_\phi(\boldsymbol{h}(t), \boldsymbol{u}(t), t) = f_\phi'(\boldsymbol{h}(t), t)\frac{dU_i}{dt}(t)$, continuously changed by the control action $\frac{dU_i}{dt}(t)$. Then, $H_{i,M}$ can be inferred by inserting the time values t_i to t_{i+M}.

Predictive-sequence controlled ODE-RNN. In ODE-RNN [51], the observations, processed by another RNN, work as a controller to sequentially change the initial state of $\boldsymbol{h}(t)$ for each interval $[t_i, t_{i+1}]$. To distinguish the built-in RNN from the discrete-time model $g_\psi(\cdot)$, we denote the cell block of the former as $g'_\phi(\cdot)$, and the subscript ϕ implies that $g'_\phi(\cdot)$ and $f_\phi(\cdot)$ have a common parameter set. Hence, the following updates exist for each $t_i \le t \le t_{i+1}$:

$$\tilde{\boldsymbol{h}}(t) = \boldsymbol{h}(t_i) + \int_{t_i}^{t} f_\phi(\boldsymbol{h}(s), \boldsymbol{u}_i, s)ds, \tag{9}$$

where control actions $\boldsymbol{u}_i$ are calculated from Equation (6). $H_{i,M}$ can be calculated from t_i to t_{i+M}.

To summarize, Equations (8) and (9) merge discrete-time model $g_\psi(\cdot)$ and continuous-time model $f_\phi(\cdot)$ to construct a coordinated model. Fig. 1 demonstrates how the two models (light/dark blue boxes) work together to output the state flow $\boldsymbol{h}(t)$ (light brown curve). Subsequently, the flow from $\boldsymbol{h}(t_i)$ to $\boldsymbol{h}(t_{i+M})$ should be evaluated with a cost function in Optimization (7), which is the core of MPC to greedily maintain optimality. Thus, we present the cost function according to different tasks.

Classification cost. For a classification problem, we define the cost as the classification loss evaluated at the terminal state at t_{i+M}. Hence, Optimization (7) drives the whole feature flow towards more separable regions. Specifically, we have:

$$J(H_{i,M}) = L_{\text{CE}}\Big(\ell^2_\phi\big(\boldsymbol{h}(t_{i+M})\big), y\Big), \tag{10}$$

where ℓ^2_ϕ is a readout neural network to map from hidden states to labels, y is the true label for input time series, and $L_{\text{CE}}(\cdot,\cdot)$ is the cross-entropy loss.

Regression cost. In a regression task, the cost is the mean square error (MSE, $L_{\text{MSE}}(\cdot,\cdot)$) for each state.

$$J(H_{i,M}) = \sum_{k=0}^{M} L_{\text{MSE}}\Big(\ell^2_\phi\big(\boldsymbol{h}(t_{i+k})\big), \boldsymbol{x}_{i+k}\Big), \tag{11}$$

where $\ell^2_\phi(\cdot)$, with a slight abuse of notation, maps from state to time-series values.

Action regularization. The norm of the action is used in MPC to mitigate control chattering and guarantee stability and convergence [5, 61]. However, this may limit the expressivity of the discrete-time model. Instead, we encourage the consistence of actions: they should all be tied to most relative features associated with the corresponding labels/values. To this end, we endow the discrete-time model with the capacity to solely complete the learning task:

$$\hat{J}(\psi) = \begin{cases} \sum_{k=0}^{M} L_{\text{CE}}\Big(\ell^2_\psi\big(\boldsymbol{u}_{i+k}\big), y\Big) \\ \qquad\qquad \text{Classification loss} \\ \sum_{k=0}^{M} L_{\text{MSE}}\Big(\ell^2_\psi\big(\boldsymbol{u}_{i+k}\big), \boldsymbol{x}_{i+k}\Big) \\ \qquad\qquad \text{Regression loss,} \end{cases} \tag{12}$$

where $\ell^2_\psi(\cdot)$ is a neural network to convert actions to labels or time-series data.

Training algorithms.As shown in Algorithm 1, NPC solves Optimization (7) by gradient descent and only conducts 1-horizon optimal control action to reach $\boldsymbol{h}(t_{i+1})$. This approach is known to provide good convergence to infinite-horizon minimization [61].

Algorithm 1 MPC-based Training Algorithm to Train NPC.

Input: A sequence of observations $\{\boldsymbol{x}_i\}_{i=1}^N$ at times $\{t_i\}_{i=1}^N$. For a classification problem, the time-series label y is also provided. There can be multiple sequences and labels.
Initialize: $\boldsymbol{z}_0 = \boldsymbol{0}$. The number of look-ahead horizons M and penalty term λ.
while Not converge **do**
  for i in $1, 2, \cdots, N$ **do**
    Compute actions $U_{i,M}$ as outputs of $g_\psi(\cdot)$ in Equation (6).
    Compute $H_{i,M}$ in Equation (8) or (9) with control actions and a continuous-time model $f_\phi(\boldsymbol{h}(t), \boldsymbol{u}(t), t)$.
    Conduct M-horizon minimization in Equation (7).
    Conduct the action $\boldsymbol{u}_{i+1}(\psi^*)$, where ψ^* is a current optimal solution.
Output: Optimal parameters (ψ^*, ϕ^*).

4 Theoretical Analysis

In this section, we provide theoretical insights about how MPC can guide the training towards a more robust and generalized solution. In particular, we elaborate on the function class of $f_\phi(\boldsymbol{h}(t), \boldsymbol{u}(t), t)$ that can be linearized into a state-space model (SSM) with a bounded error, as shown in Assumption 1 in Appendix A. Hence, linear ODE and control theorems can be used to analyze the infinite-horizon problem and derive the convergent distance between M-horizon and infinite-horizon solutions. This analysis sheds light on the general analysis, even when the derivative can be nonlinear: as shown in Equations (13) and (14), the bounded linearization error can also converge. While it's hard to directly quantify the potential bounds as different time-series data have different complexities, a large amount of work [2, 23, 24, 56, 59] reveals that SSM is highly competent for generic time-series modeling.

For a classification problem, without the loss of generality, the proposed NPC aims to start from $\boldsymbol{h}(t_1)$ and stabilize $\boldsymbol{h}(t)$ to the origin such that $\ell^2_\phi(\boldsymbol{0}) \approx C \cdot \boldsymbol{0} = \boldsymbol{0}$, implying that the label $y = 0$, where C is defined in the SSM in Assumption 1. Other labels have similar processes with some variable transformations. For a regression problem, the control goal is to enable $\ell^2_\phi(\boldsymbol{h}(t))$ to certain values. Then, we answer the following two questions: (*Q1. Stability*). Will the flow of $\boldsymbol{h}(t)$ get stable at the origin with different initial values $\boldsymbol{h}(t_1)$? (*Q2. Generalizability*). Will the flow and control sequence under M-horizon optimization converge to the infinite-horizon optimal solutions?

Theorem 1 (Stability). *Assume Assumption 1 holds. Let* $T = t_{i+M} - t_i$, $\tau = t_{i+1} - t_i$ *and* $\boldsymbol{h}_{(\psi^*,\phi^*)}(t)$ *denote the optimal state after NPC training in Algorithm 1. There exists constants* K, K_1, K_2, μ_∞, *and* $M_\infty \ge 1$ *such that*

$$||\boldsymbol{h}_{(\psi^*,\phi^*)}(t)|| \le M_\infty e^{-\mu t}||\boldsymbol{h}^*_{(\psi^*,\phi^*)}(t_1)||+ \frac{1-e^{-\mu t}}{\mu}K(1+(L+1)\tau e^{K(L+1)\tau})||\boldsymbol{w}||_{l^\infty(0,t)}, \tag{13}$$

where $\mu = \mu_\infty - K_1 e^{-2\mu_\infty(T-\tau)} - K_2 L - KL(L+1)\tau e^{K(L+1)\tau}$, $\boldsymbol{h}^*_{(\psi^*,\phi^*)(t)}$ *is the globally optimal solution, and* $||\cdot||_{l^p(0,t)}$ *is the l-p norm on the function space over* $(0,t)$.

The proof can be seen in Appendix B. When the linearization error, measured by L defined in Assumption 1, is small and $T-\tau$ is large (i.e., a large M in the M-horizon optimization), $\mu > 0$. Hence, the first term on the Right-Hand-Side (RHS) of Equation (13) exponentially decreases as t increases. The second term on RHS is limited when $\boldsymbol{w}(t)$ and L are small. Thus, $\boldsymbol{h}_{(\psi^*,\phi^*)}(t)$ gets stable to the origin exponentially. For the generalizability, we have:

Theorem 2 (Generalizability). *Assume Assumption 1 holds. Consider* T, τ, K_2, μ_∞, *and* $\boldsymbol{h}_{(\psi^*,\phi^*)}(t)$ *defined in Theorem 1 and let* $\boldsymbol{u}_{(\psi^*,\phi^*)}(t)$ *denote the optimal control action after NPC training in Algorithm 1. Let* $\boldsymbol{u}^*_\infty(t)$ *denote the optimal solution of applying the linear model in Assumption 1 to the infinite-horizon minimization problem, defined in Equation (3) in Appendix A . Let* $\boldsymbol{h}^*_\infty(t)$ *denote the state controlled by* $\boldsymbol{u}^*_\infty(t)$ *using the non-linear model* $f_{\phi^*}(\cdot)$. *There exists a constant* K_3 *such that:*

$$\begin{aligned} &||\boldsymbol{h}_{(\psi^*,\phi^*)}(t) - \boldsymbol{h}^*_\infty(t)|| + ||\boldsymbol{u}_{(\psi^*,\phi^*)}(t) - \boldsymbol{u}^*_\infty(t)|| \\ &\le K_3 e^{-2\mu_\infty(T-\tau)}\left(\frac{L+1}{\mu_\infty - K_2 L}||\boldsymbol{h}||_{l^1(0,t)} + ||\boldsymbol{h}(t)||\right) \\ &+K_3\tau e^{K_3(L+1)\tau}\frac{L+1}{\mu_\infty - K_2 L}(L||\boldsymbol{h}||_{l^1(0,t)} + ||\boldsymbol{w}||_{l^\infty(0,t)}). \end{aligned} \tag{14}$$

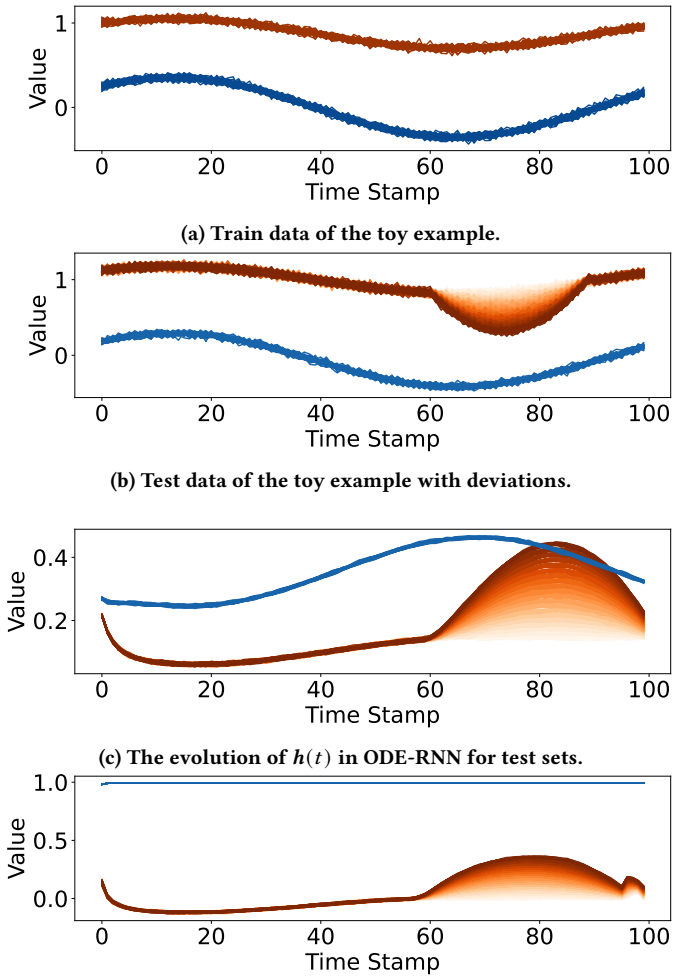
The proof is in Appendix C. By Theorem 1, $||\boldsymbol{h}(t)||$ has an exponential decay to 0. Moreover, $T-\tau$ is sufficiently large. Thus, the first term of RHS in Equation (14) decreases exponentially, and the second term is small as long as $\boldsymbol{w}(t)$ is sufficiently small, demonstrating the high generalizability.

5 Experiments

5.1 Settings

Datasets. We use the following datasets for experiments. (1) **Synthetic dataset**. We create a toy example to validate the stability of our NPC framework. The specific data generation process is described in Appendix D. Fig. 2a and 2b visualize the training and the test time series, and different colors (blue and brown) represent different labels. In the test dataset, we introduce different levels of deviations (i.e., brown colors from light to dark) to evaluate the stability. (2) **Human Activity Recognition (HAR) dataset.** HAR data [1] has recordings of 30 subjects from waist-mounted smartphone with embedded inertial sensors. Observations include linear acceleration and angular velocity, and there are 6 different types of labels describing different activities. (3) **UCR Time Series Archive.** The archive [15] contains 85 different types of time series from diverse domains. The time-series length ranges from 60 to 2700, and the number of label classes ranges from 2 to 60. We randomly select 9 datasets to test. (4) **Photovoltaic (PV) datasets.** We introduce a publicly available Photovoltaic (PV) dataset [8] about the sequential solar power generations. The values are largely determined by the continuous movement of the sun and the wind. These datasets are selected due to diversified applications, complex and continuous dynamics, and potentially irregular samples.

Benchmark methods. The following methods are utilized as benchmarks. (1) RNN-Δ_t. The time difference between every two observations, i.e., Δ_t, is introduced to a classic RNN model [12]. (2) RNN Decay (RNN-D). An exponential decay process is introduced to capture the dynamical changes of hidden states between observed timestamps in an RNN [44]. (3) ODE-RNN [51]. Neural ODE is embedded to learn the dynamical function of hidden states between every two observed timestamps. (4) Neural CDE (NCDE) [29]. Neural CDE creates a continuous data path to control the evolution of the state's ODE flow. (5) ContiFormer (ContiF.) [16]. ContiFormer generalizes Neural CDE control as a continuous attention mechanism for integrating dynamical information. **For our proposed NPC framework, we utilize an RNN as the discrete-time model and an ODE-RNN as the continuous-time model**.



(a) Train data of the toy example.

(b) Test data of the toy example with deviations.

(c) The evolution of $h(t)$ in ODE-RNN for test sets.

(d) The evolution of $h(t)$ in the proposed NPC for test sets.

Figure 2: Visualization of data and features in toy examples.

5.2 Verification of the High Stability in NPC

For the synthetic dataset in Fig. 2a and 2b, described in Setting, we visualize feature flow $\boldsymbol{h}(t)$ in for ODE-RNN and NPC, shown in Fig. 2c and 2d, respectively. We first analyze the models and observe that, for this binary classification task, ODE-RNN uses a decision boundary of **0.17**, whereas NPC adopts a more standard

Table 2: Classification test accuracy (%) with mean ± standard deviation for different baselines.

	HAR	Earth	ECG	Car	WorSyn.	Trace	Plane	Fish	Symbol	SynCon.
ODE-RNN	63.1 ± 0.09	82.3 ± 0.08	91.1 ± 0.11	66.7 ± 0.12	44.5 ± 0.10	97.0 ± 0.08	99.0 ± 0.09	64.0 ± 0.12	51.8 ± 0.13	97.3 ± 0.11
RNN-Δ_t	60.9 ± 0.11	81.1 ± 0.12	90.7 ± 0.12	46.7 ± 0.13	43.7 ± 0.14	69.0 ± 0.20	83.1 ± 0.13	65.1 ± 0.15	85.9 ± 0.11	96.7 ± 0.10
RNN-D	55.9 ± 0.18	82.0 ± 0.13	58.4 ± 0.09	21.7 ± 0.10	46.6 ± 0.12	98.0 ± 0.13	77.0 ± 0.14	74.9 ± 0.09	78.4 ± 0.08	94.3 ± 0.12
NCDE	31.8 ± 0.13	70.5 ± 0.10	75.7 ± 0.19	25.0 ± 0.14	24.5 ± 0.15	59.0 ± 0.18	41.9 ± 0.11	23.4 ± 0.09	67.4 ± 0.13	57.0 ± 0.12
Contif.	58.7 ± 0.11	82.0 ± 0.12	**93.9 ± 0.14**	21.7 ± 0.21	21.9 ± 0.11	49.0 ± 0.17	96.2 ± 0.14	12.6 ± 0.10	85.6 ± 0.11	89.7 ± 0.13
NPC	**70.1 ± 0.11**	**85.3 ± 0.09**	91.1 ± 0.12	**76.7 ± 0.10**	**50.6 ± 0.11**	**99.8 ± 0.08**	**99.7 ± 0.11**	**77.7 ± 0.09**	**86.5 ± 0.11**	**99.9 ± 0.07**

threshold of **0.5**. Comparing these boundaries with the end-point feature values (i.e., $\boldsymbol{h}(t_{100})$), it's clear that the NPC classifier has a larger classification margin that leads to more robust results.

More specifically, for ODE-RNN's result in Fig. 2c, the deviations of test data cause $\boldsymbol{h}(t)$ to change with a high sensitivity. With test deviations, some feature flow's (brown lines) end-point feature value is larger than 0.17 and wrongly labeled as blue, leading to 87.2% test accuracy. For our NPC's result in Fig. 2d, in the beginning, the blue and the brown features are quickly set apart and converge to equilibrium points, i.e., 1.0 and 0.0, respectively. This is because NPC looks ahead with the current dynamics and concludes that features should get stable to equilibrium points to minimize the error. When disturbances appear at timestamp 60, the impacted features have much smaller deviations compared to those of DOE-RNN, implying that it's harder to leave the equilibrium point. Consequently, the brown endpoints are smaller than the boundary, i.e., 0.5, yielding 100% test accuracy.

5.3 Stability Guarantees General Classification Improvements on Diversified Domains

We evaluate the overall performance for time-series classification. 80% of the data is randomly dropped to create irregularly sampled observations. Then, Table 2 demonstrates the result. In most cases, our NPC has an accuracy increase of 2% ∼ 15%, compared to the state-of-the-art. In particular, when the train/test data have a significant distribution discrepancy, like HAR, CAR, and WorSyn., our methods perform much better (6% ∼ 15%), which indicates the robustness against the data deviations due to the high stability. For the dataset ECG5000, ContiFormer performs slightly better. This could happen because our tested NPC is based on ODE-RNN, which may underperform ContiFormer when the time series is relatively long. However, ContiFormer can be utilized in our NPC framework.

5.4 Generalizability Leads to Accurate Interpolation and Extrapolation

Next, we test regression tasks, i.e., interpolation and extrapolation, on PV datasets with a drop rate in {40%, 60%, 80%}. When dropping 80% data, Fig. 4 visualizes the interpolated and the true data using ODE-RNN and our NPC for PV datasets for several hours, where the x-axis unit is minutes. Obviously, the interpolated data from NPC are much closer to the ground truth. This is because, in NPC, the additional discrete RNN adjusts the interpolation of ODE-RNN by minimizing the M-horizon predictions. This gives much richer information for current interpolation and approximates the infinite-horizon result. Table 3 exhibits averaged results for different methods. NPC gains consistent and significant improvements under various drop rates. Moreover, we plot the extrapolation results for PV and load datasets in Fig. 3. They cover a week's data, where the x-axis unit is hours. The results demonstrate that NPC methods can successfully approximate the short-term dynamics and predict the most accurate results. In particular, we note that from 120h to 168h (i.e., the weekends), load and PV have gone through a distributional shift. Under this condition, our discrete-time model in NPC can sense the context change and adaptively change the flow evolution in the continuous-time model, thus leading to accurate results. For example, we can observe different dynamics between weekdays and weekends.

Table 3: Interpolation and extrapolation results on PV datasets.

Drop Rate	Metric	ODE-RNN	RNN-Δ_t	RNN-D	Contif.	NPC
Interpolation						
40%	RMSE	0.044	0.080	0.075	0.051	**0.037**
	MAPE (%)	2.13	3.89	3.76	2.66	**1.79**
60%	RMSE	0.058	0.093	0.068	0.064	**0.041**
	MAPE (%)	2.94	4.10	3.28	3.21	**2.21**
80%	RMSE	0.060	0.110	0.087	0.069	**0.051**
	MAPE (%)	3.06	4.54	4.17	3.44	**2.37**
Extrapolation						
40%	RMSE	0.071	0.088	0.092	0.033	**0.020**
	MAPE (%)	4.17	4.86	5.21	2.25	**1.65**
60%	RMSE	0.075	0.094	0.098	0.048	**0.026**
	MAPE (%)	4.46	5.22	5.51	3.04	**1.98**
80%	RMSE	0.089	0.120	0.100	0.055	**0.038**
	MAPE (%)	4.86	6.53	5.73	3.19	**2.41**

5.5 Sensitivity and Efficiency Analysis

We conduct sensitivity analysis with respect to the horizon number M. We utilize the interpolation problem as an example and vary $M \in \{2, \cdots, 8\}$. Fig 5 illustrates the RMSE with respect to M for the NPC method under different data drop rates. At first glance, the results seemingly violate the theory that the larger M is, the better. In particular, we can observe that the optimal M in Fig 5 shifts to the right as the drop rate decreases. We attribute the phenomena to the fact that the denser the data are, the easier it is to learn $f_\phi(\cdot)$. Hence, for dense time series, we can assign a larger M to attain convergence in Theorem 1 and 2. However, for sparse data,

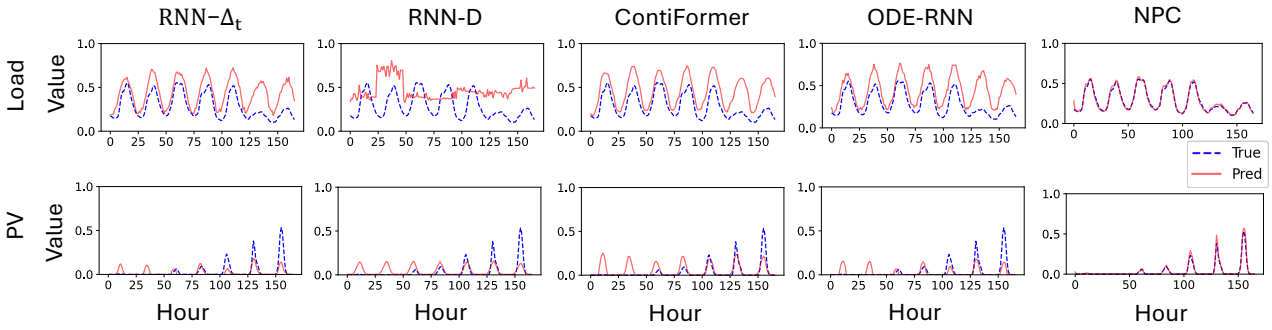
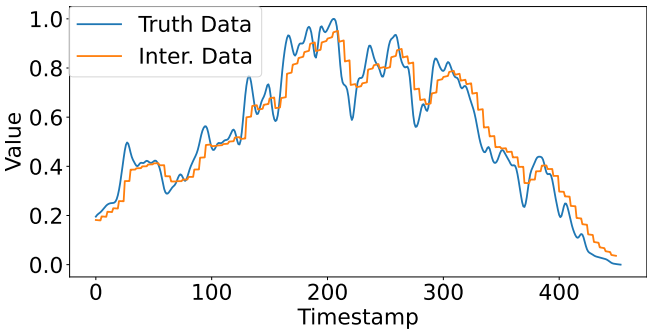
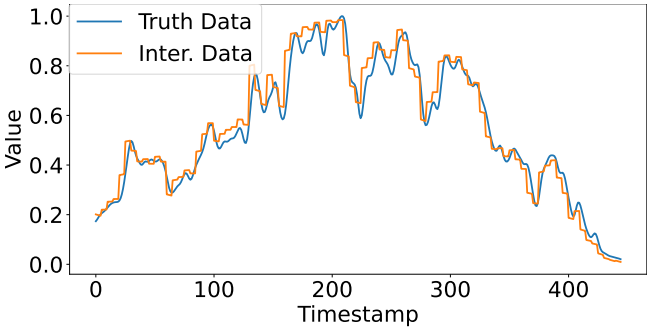


Figure 3: Extrapolation results for 168h data in a week.



(a) Interpolation result of ODE-RNN.



(b) Interpolation result of NPC.

Figure 4: Interpolation results for 420min data.

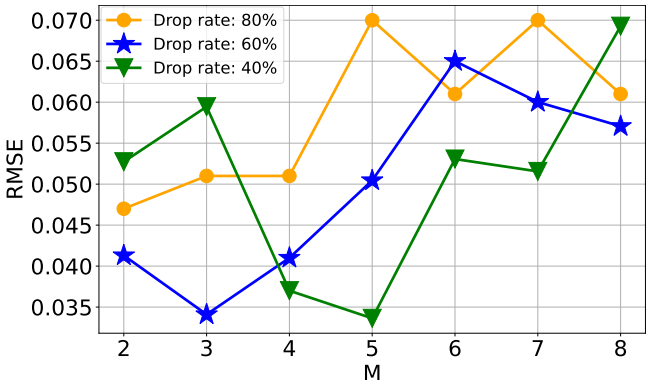


Figure 5: The sensitivity analysis

ContiFormer needs to compute expensive continuous attention and has a much longer test time.

Table 4: Test time (s) for different methods.

Method	RNN-Δ_t	RNN-D	ODE-RNN	NCDE	ContiF.	NPC
Time (s)	0.033	0.031	0.121	0.034	3.609	0.389

if M is too large, the M-horizon minimization and the learning $f_\phi(\cdot)$ negatively affect each other at initial iterations due to random parameter initializations. This suggests a novel future direction to improve the training algorithm.

In the training phase, compared to ODE-RNN, NPC requires around $N(M-1)$ more iterations for solving an ODE. Improving the training efficiency is listed as future work in Conclusion. However, in the testing phase, only 1 out of M action is conducted each time. Hence, NPC has comparable test efficiency. For example, we consider the Car dataset with $(60, 73)$ sample numbers and length, respectively. The test time is listed in Table 4. NPC is around three times that of ODE-RNN but still affordable for real-time predictions.

6 Related Work

Control Theory-based Deep Learning Models. Several recent studies have shed light on applying insightful control theories to DL. Inspired by the fact that Resnet [25] and Neural ODE are dynamical functions, their training procedure becomes an optimal control problem where the parameters of neural networks are control variables [50]. Novel training methods are proposed according to mature control analyses like maximum principle [7, 38, 57, 64], mean-field theory [39, 62], feedback control [11], and Lyapunov analysis [28, 50]. However, these analyses are hardly applicable to time series. In particular, sequential control with temporal information is needed. To chase optimal sequential control and simultaneously guarantee stability or generalizability, MPC is a natural choice.

Deep Learning for Non-linear Control Systems. There is a different domain that exploits well-trained DL models for non-linear control in physical systems. For instance, the total deviations from a desired trajectory should be minimized in a vehicle trajectory tracking problem [65]. In these systems, DL models are utilized to approximate unknown system dynamics for MPC formulations [13, 41, 48, 65]. This suggests the high potential of combining MPC and DL.

7 Conclusion, Limitation, and Future Work

We propose NPC, a novel method to coordinate arbitrary discrete- and continuous-time DL models to efficiently utilize short-term dynamics and long-term patterns. Moreover, our model can provably achieve high stability and generalizability. The target is an infinite-horizon cost minimization. Grounded in the optimal control theory, we apply the finite-horizon relaxation with an exponential convergence. Furthermore, a feedback mechanism is added as additional information to improve the convergence. The framework is simple yet highly effective, gaining the best performance on various time-series tasks. The limitation of NPC is the high training time due to the additional $O(N \cdot M)$ ODE computations and minimization. In the future, we could address the issue by (1) selectively conducting the M-horizon optimization and (2) employing SSM instead of Neural ODE to capture the dynamics in the continuous-time model. Then, fast ODE inferences could be achieved based on Kalman filter [2, 56] or Fast Fourier Transform (FFT) [20].

Acknowledgments

This work is supported by NSF 2433308, NSF 2048288 and AFOSR FA9550-22-1-0294.

References

[1] Davide Anguita, Alessandro Ghio, Luca Oneto, Xavier Parra, Jorge Luis Reyes-Ortiz, et al. 2013. A public domain dataset for human activity recognition using smartphones.. In *Esann*, Vol. 3. 3.

[2] Abdul Fatir Ansari, Alvin Heng, Andre Lim, and Harold Soh. 2023. Neural continuous-discrete state space models for irregularly-sampled time series. In *International Conference on Machine Learning*. PMLR, 926–951.

[3] Thomas Asikis, Lucas Böttcher, and Nino Antulov-Fantulin. 2022. Neural ordinary differential equation control of dynamics on graphs. *Physical Review Research* 4, 1 (2022), 013221.

[4] John M Ball. 1977. Strongly continuous semigroups, weak solutions, and the variation of constants formula. *Proc. Amer. Math. Soc.* 63, 2 (1977), 370–373.

[5] Federico Pizarro Bejarano, Lukas Brunke, and Angela P Schoellig. 2023. Multi-Step Model Predictive Safety Filters: Reducing Chattering by Increasing the Prediction Horizon. In *2023 62nd IEEE Conference on Decision and Control (CDC)*. IEEE, 4723–4730.

[6] Peter Benner and Hermann Mena. 2018. Numerical solution of the infinite-dimensional LQR problem and the associated Riccati differential equations. *Journal of Numerical Mathematics* 26, 1 (2018), 1–20.

[7] Martin Benning, Elena Celledoni, Matthias J Ehrhardt, Brynjulf Owren, and Carola-Bibiane Schönlieb. 2019. Deep learning as optimal control problems: Models and numerical methods. *arXiv preprint arXiv:1904.05657* (2019).

[8] Matthew Boyd. 2016. NIST Weather Station for Photovoltaic and Building System Research. *National Institute of Standards and Technology, Gaithersburg, MD, Technical Note* 1913 (2016).

[9] Lucian Busoniu, Robert Babuska, Bart De Schutter, and Damien Ernst. 2017. *Reinforcement learning and dynamic programming using function approximators*. CRC press.

[10] Frank M Callier, Joseph Winkin, and Jacques L Willems. 1994. Convergence of the time-invariant Riccati differential equation and LQ-problem: mechanisms of attraction. *International journal of control* 59, 4 (1994), 983–1000.

[11] Mathieu Chalvidal, Matthew Ricci, Rufin VanRullen, and Thomas Serre. 2020. Go with the flow: Adaptive control for neural ODEs. *arXiv preprint arXiv:2006.09545* (2020).

[12] Zhengping Che, Sanjay Purushotham, Kyunghyun Cho, David Sontag, and Yan Liu. 2018. Recurrent neural networks for multivariate time series with missing values. *Scientific reports* 8, 1 (2018), 6085.

[13] Kong Yao Chee, Tom Z Jiahao, and M Ani Hsieh. 2022. Knode-mpc: A knowledge-based data-driven predictive control framework for aerial robots. *IEEE Robotics and Automation Letters* 7, 2 (2022), 2819–2826.

[14] Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. 2018. Neural ordinary differential equations. *Advances in neural information processing systems* 31 (2018).

[15] Yanping Chen, Eamonn Keogh, Bing Hu, Nurjahan Begum, Anthony Bagnall, Abdullah Mueen, and Gustavo Batista. 2015. The UCR Time Series Classification Archive. www.cs.ucr.edu/~eamonn/time_series_data/.

[16] Yuqi Chen, Kan Ren, Yansen Wang, Yuchen Fang, Weiwei Sun, and Dongsheng Li. 2024. ContiFormer: Continuous-time transformer for irregular time series modeling. *Advances in Neural Information Processing Systems* 36 (2024).

[17] Junyoung Chung, Caglar Gulcehre, KyungHyun Cho, and Yoshua Bengio. 2014. Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint arXiv:1412.3555* (2014).

[18] Edward De Brouwer, Jaak Simm, Adam Arany, and Yves Moreau. 2019. GRU-ODE-Bayes: Continuous modeling of sporadically-observed time series. *Advances in neural information processing systems* 32 (2019).

[19] Emilien Dupont, Arnaud Doucet, and Yee Whye Teh. 2019. Augmented neural odes. *Advances in neural information processing systems* 32 (2019).

[20] Daniel Y Fu, Tri Dao, Khaled K Saab, Armin W Thomas, Atri Rudra, and Christopher Ré. 2022. Hungry hungry hippos: Towards language modeling with state space models. *arXiv preprint arXiv:2212.14052* (2022).

[21] Weilong Fu and Ali Hirsa. 2022. An unsupervised deep learning approach to solving partial integro-differential equations. *Quantitative Finance* 22, 8 (2022), 1481–1494.

[22] Carlos E Garcia, David M Prett, and Manfred Morari. 1989. Model predictive control: Theory and practice—A survey. *Automatica* 25, 3 (1989), 335–348.

[23] Albert Gu, Karan Goel, Ankit Gupta, and Christopher Ré. 2022. On the parameterization and initialization of diagonal state space models. *Advances in Neural Information Processing Systems* 35 (2022), 35971–35983.

[24] Albert Gu, Karan Goel, and Christopher Ré. 2021. Efficiently modeling long sequences with structured state spaces. *arXiv preprint arXiv:2111.00396* (2021).

[25] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.

[26] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation* 9, 8 (1997), 1735–1780.

[27] Sheo Yon Jhin, Heejoo Shin, Sujie Kim, Seoyoung Hong, Minju Jo, Solhee Park, Noseong Park, Seungbeom Lee, Hwiyoung Maeng, and Seungmin Jeon. 2024. Attentive neural controlled differential equations for time-series classification and forecasting. *Knowledge and Information Systems* 66, 3 (2024), 1885–1915.

[28] Qiyu Kang, Yang Song, Qinxu Ding, and Wee Peng Tay. 2021. Stable neural ode with lyapunov-stable equilibrium points for defending against adversarial attacks. *Advances in Neural Information Processing Systems* 34 (2021), 14925–14937.

[29] Patrick Kidger, James Morrill, James Foster, and Terry Lyons. 2020. Neural controlled differential equations for irregular time series. *Advances in Neural Information Processing Systems* 33 (2020), 6696–6707.

[30] Basil Kouvaritakis and Mark Cannon. 2016. Model predictive control. *Switzerland: Springer International Publishing* 38 (2016), 13–56.

[31] Venkataramanaiah Krishnamurthy. 2019. Predictability of weather and climate. *Earth and Space Science* 6, 7 (2019), 1043–1056.

[32] Peter Lancaster and Leiba Rodman. 1995. *Algebraic riccati equations*. Clarendon press.

[33] Haoran Li, Muhao Guo, Marija Ilic, Yang Weng, and Guangchun Ruan. 2025. External Data-Enhanced Meta-Representation for Adaptive Probabilistic Load Forecasting. *arXiv preprint arXiv:2506.23201* (2025).

[34] Haoran Li, Muhao Guo, Yang Weng, Marija Ilic, and Guangchun Ruan. 2025. ExARNN: An Environment-Driven Adaptive RNN for Learning Non-Stationary Power Dynamics. *arXiv preprint arXiv:2505.17488* (2025).

[35] Haoran Li, Zhihao Ma, and Yang Weng. 2022. A Transfer Learning Framework for Power System Event Identification. *IEEE Transactions on Power Systems* 37, 6 (2022), 4424–4435. doi:10.1109/TPWRS.2022.3153445

[36] Haoran Li, Zhihao Ma, Yang Weng, Haiwang Zhong, and Xiaodong Zheng. 2025. Low-Dimensional ODE Embedding to Convert Low-Resolution Meters Into "Virtual" PMUs. *IEEE Transactions on Power Systems* 40, 2 (2025), 1439–1451. doi:10.1109/TPWRS.2024.3427637

[37] Haoran Li and Yang Weng. 2023. PIX-GAN: Enhance Physics-Informed Estimation via Generative Adversarial Network. In *2023 IEEE International Conference on Data Mining (ICDM)*. 1085–1090. doi:10.1109/ICDM58522.2023.00128

[38] Qianxiao Li, Long Chen, Cheng Tai, and E Weinan. 2018. Maximum principle based algorithms for deep learning. *Journal of Machine Learning Research* 18, 165 (2018), 1–29.

[39] Guan-Horng Liu and Evangelos A Theodorou. 2019. Deep learning theory review: An optimal control and dynamical systems perspective. *arXiv preprint*

arXiv:1908.10920 (2019).
[40] Santíago V Luis and Eduardo García-Verdugo. 2010. *Chemical reactions and processes under flow conditions*. Vol. 5. Royal Society of Chemistry.
[41] Junwei Luo, Fahim Abdullah, and Panagiotis D Christofides. 2023. Model predictive control of nonlinear processes using neural ordinary differential equation models. *Computers & Chemical Engineering* 178 (2023), 108367.
[42] Stefano Massaroli, Michael Poli, Jinkyoo Park, Atsushi Yamashita, and Hajime Asama. 2020. Dissecting Neural ODEs. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 3952–3963. https://proceedings.neurips.cc/paper_files/paper/2020/file/293835c2cc75b585649498ee74b395f5-Paper.pdf
[43] James Morrill, Cristopher Salvi, Patrick Kidger, and James Foster. 2021. Neural rough differential equations for long time series. In *International Conference on Machine Learning*. PMLR, 7829–7838.
[44] Michael C Mozer, Denis Kazakov, and Robert V Lindsey. 2017. Discrete event, continuous time rnns. *arXiv preprint arXiv:1710.04110* (2017).
[45] Dorota Mozyrska, Ewa Pawluszewicz, and Delfim FM Torres. 2009. The Riemann-Stieltjes integral on time scales. *arXiv preprint arXiv:0903.1224* (2009).
[46] Hoang Nguyen, Le-Minh Kieu, Tao Wen, and Chen Cai. 2018. Deep learning methods in transportation domain: a review. *IET Intelligent Transport Systems* 12, 9 (2018), 998–1004.
[47] Bernhard Ø Palsson. 2011. *Systems biology: simulation of dynamic network states*. Cambridge University Press.
[48] Yunpeng Pan and Jun Wang. 2011. Model predictive control of unknown nonlinear dynamical systems based on recurrent neural networks. *IEEE Transactions on Industrial Electronics* 59, 8 (2011), 3089–3101.
[49] Alessio Porretta and Enrique Zuazua. 2013. Long time versus steady state optimal control. *SIAM Journal on Control and Optimization* 51, 6 (2013), 4242–4273.
[50] Ivan Dario Jimenez Rodriguez, Aaron Ames, and Yisong Yue. 2022. Lyanet: A lyapunov framework for training neural odes. In *International Conference on Machine Learning*. PMLR, 18687–18703.
[51] Yulia Rubanova, Ricky TQ Chen, and David K Duvenaud. 2019. Latent ordinary differential equations for irregularly-sampled time series. *Advances in neural information processing systems* 32 (2019).
[52] Domenec Ruiz-Balet and Enrique Zuazua. 2023. Neural ODE control for classification, approximation, and transport. *SIAM Rev.* 65, 3 (2023), 735–773.
[53] Andrew P Sage. 1968. Optimum systems control. *(No Title)* (1968).
[54] Mainak Sarkar and Arnaud De Bruyn. 2021. LSTM response models for direct marketing analytics: Replacing feature engineering with deep learning. *Journal of Interactive Marketing* 53, 1 (2021), 80–95.
[55] Michael Scheutzow. 2013. A stochastic Gronwall lemma. *Infinite Dimensional Analysis, Quantum Probability and Related Topics* 16, 02 (2013), 1350019.
[56] Mona Schirmer, Mazin Eltayeb, Stefan Lessmann, and Maja Rudolph. 2022. Modeling irregular time series with continuous recurrent units. In *International Conference on Machine Learning*. PMLR, 19388–19405.
[57] Jacob H Seidman, Mahyar Fazlyab, Victor M Preciado, and George J Pappas. 2020. Robust deep learning as optimal control: Insights and convergence guarantees. In *Learning for Dynamics and Control*. PMLR, 884–893.
[58] Ikaro Silva, George Moody, Daniel J Scott, Leo A Celi, and Roger G Mark. 2012. Predicting in-hospital mortality of icu patients: The physionet/computing in cardiology challenge 2012. In *2012 computing in cardiology*. IEEE, 245–248.
[59] Jimmy TH Smith, Andrew Warrington, and Scott W Linderman. 2022. Simplified state space layers for sequence modeling. *arXiv preprint arXiv:2208.04933* (2022).
[60] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
[61] Daniel Veldman and Enrique Zuazua. 2022. Local Stability and Convergence of Unconstrained Model Predictive Control. *arXiv preprint arXiv:2206.01097* (2022).
[62] E Weinan, Jiequn Han, and Qianxiao Li. 2018. A mean-field optimal control formulation of deep learning. *arXiv preprint arXiv:1807.01083* (2018).
[63] Qingsong Wen, Tian Zhou, Chaoli Zhang, Weiqi Chen, Ziqing Ma, Junchi Yan, and Liang Sun. 2022. Transformers in time series: A survey. *arXiv preprint arXiv:2202.07125* (2022).
[64] Dinghuai Zhang, Tianyuan Zhang, Yiping Lu, Zhanxing Zhu, and Bin Dong. 2019. You only propagate once: Accelerating adversarial training via maximal principle. *Advances in neural information processing systems* 32 (2019).
[65] Kunwu Zhang, Qi Sun, and Yang Shi. 2021. Trajectory tracking control of autonomous ground vehicles using adaptive learning MPC. *IEEE Transactions on Neural Networks and Learning Systems* 32, 12 (2021), 5554–5564.
[66] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. 2021. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 11106–11115.

A Assumption

Assumption 1. *Let $\frac{d\boldsymbol{h}(t)}{dt} = f(\boldsymbol{h}(t), \boldsymbol{u}(t))$ denote the true dynamics of $\boldsymbol{h}(t)$ in our Coordinated model. Consider linearization $f(\boldsymbol{h}(t), \boldsymbol{u}(t)) \approx A\boldsymbol{h}(t) + B\boldsymbol{u}(t)$ and $\ell_\phi^2(\boldsymbol{h}(t)) \approx C\boldsymbol{h}(t)$, where A, B, and C are the state, input, and output matrices of the state-space model (SSM), respectively. Assume the following conditions hold:*

- *Our approximation for the true dynamics satisfies $f_{\phi^*}(\boldsymbol{h}(t), \boldsymbol{u}(t)) = f(\boldsymbol{h}(t), \boldsymbol{u}(t)) + \boldsymbol{w}(t)$, where $\boldsymbol{w}(t)$ is an approximation error.*
- *The optimal control in NPC training is sufficiently close to the M-horizon optimal control results in the SSM model.*
- *(A, B) is controllable and (A, C) is observable.*
- *There exists a Lipschitz constant L such that $\forall i, j > 0$,*

$$||f(\boldsymbol{h}(t_i), \boldsymbol{u}(t_j)) - A\boldsymbol{h}(t_i) - B\boldsymbol{u}(t_j) - f(\boldsymbol{h}(t_j), \boldsymbol{u}(t_j)) + A\boldsymbol{h}(t_j) + B\boldsymbol{u}(t_j)|| \leq L(||\boldsymbol{h}(t_i) - \boldsymbol{h}(t_j)|| + ||\boldsymbol{u}(t_i) - \boldsymbol{u}(t_j)||).$$

The first condition depends on the approximation power of the continuous-time model can improve the approximation to make $\boldsymbol{w}(t)$ sufficiently small. The second to the fourth conditions require a small linearization error from $f(\cdot)$ to an SSM. A large amount of work [2, 23, 24, 56, 59] reveals that SSM is competent.

B Theorem 1 and Proof

Theorem (Stability). *Let $T = t_{i+M} - t_i$, $\tau = t_{i+1} - t_i$ and $\boldsymbol{h}^*_{(\psi^*,\phi^*)}(t)$ denote the trajectory after NPC training and control. There exists constants K, K_1, K_2, μ_∞, and $M_\infty \geq 1$ such that*

$$||\boldsymbol{h}_{(\psi^*,\phi^*)}(t)|| \leq M_\infty e^{-\mu t}||\boldsymbol{h}^*_{(\psi^*,\phi^*)}(t_1)|| + \frac{1 - e^{-\mu t}}{\mu}K(1 + (L+1)\tau e^{K(L+1)\tau})||\boldsymbol{w}||_{l^\infty(0,t)}, \tag{15}$$

where $\mu = \mu_\infty - K_1 e^{-2\mu_\infty(T-\tau)} - K_2 L - KL(L+1)\tau e^{K(L+1)\tau}$ and $||\cdot||_{l^\infty(0,t)}$ is the infinity norm on the function space over $(0, t)$.

Proof. To begin with, we introduce preliminary results about the infinite-horizon and M-horizon MPC for SSM. Then, we conduct the stability and convergence analysis for nonlinear dynamical equations. First, by the SSM formula in Assumption 1, we consider a continuous extension of the M-horizon minimization in Equation 6 in $[t_i, t_i + T]$:

$$J_T(\boldsymbol{u}; t_i) = \frac{1}{2}\int_{t_i}^{t_i+T} ||C\boldsymbol{h}(t)||^2 + (\boldsymbol{u}(t))^\top R\boldsymbol{u}(t)dt, \tag{16}$$

where $||C\boldsymbol{h}(t)||^2$ is the continuous version of $J(\boldsymbol{h}(t_i), \cdots, \boldsymbol{h}(t_{i+M}))$ to force $C\boldsymbol{h}(t) \to \boldsymbol{0}$ so that the feature state is classified to have label $y = 0$. $(\boldsymbol{u}(t))^\top R\boldsymbol{u}(t)$ is a continuous version of $\lambda\hat{J}(\boldsymbol{u}_i, \cdots, \boldsymbol{u}_{i+M})$ and R is a symmetric positive definite matrix. Similarly, for the infinite-horizon problem [61], we have:

$$J_\infty(\boldsymbol{u}) = \frac{1}{2}\int_{t_1}^{\infty} ||C\boldsymbol{h}(t)||^2 + (\boldsymbol{u}(t))^\top R\boldsymbol{u}(t)dt, \tag{17}$$

where t_1 is the start time in our time-series observations and we can set $t_1 = 0$ without loss of generality. By Assumption 1 in the main paper, in the SSM, the above optimizations have the following constraints:

$$\begin{aligned} \frac{d\boldsymbol{h}(t)}{dt} &= A\boldsymbol{h}(t) + B\boldsymbol{u}(t), \\ \ell_\phi^2(\boldsymbol{h}(t)) &= C\boldsymbol{h}(t), \end{aligned} \tag{18}$$

where (A, B) is controllable and (A, C) is observable. Hence, for the objective in Equation (17) and the constraint (18), the optimal trajectory of [53] is given by:

$$\begin{aligned} \frac{d\tilde{\boldsymbol{h}}^*_\infty(t)}{dt} &= A_\infty \tilde{\boldsymbol{h}}^*_\infty(t), \tilde{\boldsymbol{h}}^*_\infty(t_1) = \boldsymbol{h}(t_1), \\ A_\infty &= A - BR^{-1}B^\top P_\infty, \\ \tilde{\boldsymbol{u}}^*_\infty(t) &= -R^{-1}B^\top P_\infty \tilde{\boldsymbol{h}}^*_\infty(t), \end{aligned} \tag{19}$$

where P_∞ is the unique symmetric positive-definite solution of the following Algebraic Riccati Equation (ARE) [32]:

$$A^\top P_\infty + P_\infty A - P_\infty BR^{-1}B^\top P_\infty + C^\top C = \mathbf{0}. \tag{20}$$

According to [49], the controllability of (A, B) leads to the fact that there are constants $\mu_\infty > 0$ and $M_\infty \geq 1$ such that

$$\forall t \geq t_1, ||e^{A_\infty t}|| \leq M_\infty e^{-\mu_\infty t}, \tag{21}$$

where $||A||$ for an operator A is the operator norm. The exponentially decreasing upper bound in Equation (21) implies that if $t \to \infty$, the solution in Equation (19), i.e., $\tilde{\boldsymbol{h}}^*_\infty(t) \to \mathbf{0}$. This implies that the infinite-horizon problem has exponential convergence to the origin.

Then, for the M-horizon problem in Equation (16), by [53], the optimal trajectory is:

$$\begin{aligned} \frac{d\tilde{\boldsymbol{h}}^*_T(t)}{dt} &= A_{T,\tau}(t)\tilde{\boldsymbol{h}}^*_T(t), \tilde{\boldsymbol{h}}^*_T(t_1) = \boldsymbol{h}(t_1), \\ A_{T,\tau} &= A - BR^{-1}B^\top P(T - (t \bmod \tau)), \\ \tilde{\boldsymbol{u}}^*_T(t) &= -R^{-1}B^\top P(T - (t \bmod \tau))\tilde{\boldsymbol{h}}^*_T(t), \end{aligned} \tag{22}$$

$A_{T,\tau}$ is a τ-periodic matrix since the MPC only updates the optimal action from Equation (16) and evolve the optimal state for an interval of τ (i.e., conduct one-horizon action in our Algorithm 1 in the main paper). Subsequently, Equation (16) needs to be resolved. Essentially, $P(t)$ follows the so-called Ricatti Differential Equation (RDE) [6] in $[0, T]$:

$$\begin{aligned} \frac{dP(t)}{dt} &= A^\top P(t) + P(t)A - P(t)BR^{-1}B^\top P(t) + C^\top C, \\ P(0) &= C. \end{aligned} \tag{23}$$

To analyze the convergence of $\tilde{\boldsymbol{h}}^*_T(t)$, it follows that we need to understand the relations between the RDE solution $P(t)$ and the ARE solution P_∞. By [10, 49, 61], there exists a constant K_0 such that

$$||P(t) - P_\infty|| \leq K_0 e^{-2\mu_\infty t}. \tag{24}$$

Therefore, as $t \to \infty$, $P(t) \to P_\infty$ and $A_{T,\tau}(t) \to A_\infty$ when $T - \tau \to \infty$. This connects the convergence analysis between infinite-horizon to M-horizon results with SSM as the ODE model.

Instead of the SSM in Equation (18), in our NPC framework, we note that the constraint should satisfy:

$$\frac{d\boldsymbol{h}(t)}{dt} = f_{\phi^*}(\boldsymbol{h}(t), \boldsymbol{u}(t)) = f(\boldsymbol{h}(t), \boldsymbol{u}(t)) + \boldsymbol{w}(t), \tag{25}$$

where $\boldsymbol{w}(t)$ is a sufficiently small approximation error by the first condition in Assumption 1. For our NPC training and by the second condition in Assumption 1, we have:

$$\begin{aligned} \frac{d\boldsymbol{h}^*_{(\psi^*,\phi^*)}(t)}{dt} &= f(\boldsymbol{h}^*_{(\psi^*,\phi^*)}(t), \tilde{\boldsymbol{u}}^*_T(t)) + \boldsymbol{w}(t) \\ &= f(\boldsymbol{h}^*_{(\psi^*,\phi^*)}(t), \tilde{\boldsymbol{u}}^*_T(t)) \\ &+ \boldsymbol{w}(t) - A\boldsymbol{h}^*_{(\psi^*,\phi^*)}(t) - B\tilde{\boldsymbol{u}}^*_T(t) \\ &- BR^{-1}B^\top P(T - (t \bmod \tau))\tilde{\boldsymbol{h}}^*_T(t) + \\ &BR^{-1}B^\top P(T - (t \bmod \tau))\boldsymbol{h}^*_{(\psi^*,\phi^*)}(t) \\ &+ A_{T,\tau}(t)\boldsymbol{h}^*_{(\psi^*,\phi^*)}(t) - A_\infty \boldsymbol{h}^*_{(\psi^*,\phi^*)}(t) \\ &+ A_\infty \boldsymbol{h}^*_{(\psi^*,\phi^*)}(t) \\ &= f(\boldsymbol{h}^*_{(\psi^*,\phi^*)}(t), \tilde{\boldsymbol{u}}^*_T(t)) - A\boldsymbol{h}^*_{(\psi^*,\phi^*)}(t) \\ &- B\tilde{\boldsymbol{u}}^*_T(t) + \boldsymbol{w}(t) + A_\infty \boldsymbol{h}^*_{(\psi^*,\phi^*)}(t) \\ &+ (A_{T,\tau}(t) - A_\infty)\boldsymbol{h}^*_{(\psi^*,\phi^*)}(t) - \\ &BR^{-1}B^\top P(T - (t \bmod \tau))\boldsymbol{\epsilon}(t), \end{aligned} \tag{26}$$

where $\boldsymbol{\epsilon}(t) = \tilde{\boldsymbol{h}}^*_T(t) - \boldsymbol{h}^*_{(\psi^*,\phi^*)}(t)$. Equation (26) links the $\boldsymbol{h}^*_{(\psi^*,\phi^*)}(t)$ and $\tilde{\boldsymbol{h}}^*_T(t)$ so that the convergence of $\boldsymbol{h}^*_{(\psi^*,\phi^*)}(t)$ can be analyzed. Specifically, by the forth condition of Assumption 1 , we have:

$$\begin{aligned} &||f(\boldsymbol{h}^*_{(\psi^*,\phi^*)}(t), \tilde{\boldsymbol{u}}^*_T(t)) - A\boldsymbol{h}^*_{(\psi^*,\phi^*)}(t) - B\tilde{\boldsymbol{u}}^*_T(t)|| \\ &\leq L(||\boldsymbol{h}^*_{(\psi^*,\phi^*)}(t)|| + ||\tilde{\boldsymbol{u}}^*_T(t)||) \\ &= L(||\boldsymbol{h}^*_{(\psi^*,\phi^*)}(t)|| + \\ &|| - R^{-1}B^\top P(T - (t \bmod \tau))(\boldsymbol{\epsilon}(t) + \boldsymbol{h}^*_{(\psi^*,\phi^*)}(t))||) \\ &\leq L((1 + K_2')||\boldsymbol{h}^*_{(\psi^*,\phi^*)}(t)|| + K_2'||\boldsymbol{\epsilon}(t)||), \end{aligned} \tag{27}$$

where $K_2' = ||R^{-1}B^\top||(K_0 + ||P_\infty||)$ and the last inequality holds by the fact that $||P(T - (t \bmod \tau))|| \leq K_0 + ||P_\infty||$ from Equation (24). Applying the variation of constants formula [4] to Equation (26) and the norm operations, by Equations (21) and (27), we have:

$$\begin{aligned} ||\boldsymbol{h}^*_{(\psi^*,\phi^*)}(t)|| &\leq M_\infty e^{-\mu_\infty t}||\boldsymbol{h}^*_{(\psi^*,\phi^*)}(t_1)|| + \\ &(K_1 e^{-2\mu_\infty (T-\tau)} + K_2 L)\int_0^t e^{-\mu_\infty (t-s)} \\ &||\boldsymbol{h}^*_{(\psi^*,\phi^*)}(s)||ds \\ &+ K(L+1)\int_0^t e^{-\mu_\infty (t-s)}||\boldsymbol{\epsilon}(s)||ds \\ &+ M_\infty \int_0^t e^{-\mu_\infty (t-s)}||\boldsymbol{w}(s)||ds, \end{aligned} \tag{28}$$

where $K_2 = M_\infty(1 + K_2')$ and $K_1 = M_\infty||BR^{-1}B^\top||K_0$. Note that the above inequality also uses the inequality: $\max_t ||A_{T,\tau}(t) - A_\infty|| \leq ||BR^{-1}B^\top||K_0 e^{-2\mu_\infty (T-\tau)}$ by Equation (24). To investigate the impact of $||\boldsymbol{\epsilon}(t)||$ in Equation (28), the definition of $\boldsymbol{\epsilon}(t)$ and Equation (26) imply that:

$$\begin{aligned} \frac{d\boldsymbol{\epsilon}(t)}{dt} &= A_\infty \boldsymbol{\epsilon}(t) + (A_{T,\tau}(t) - A_\infty)\boldsymbol{\epsilon}(t) \\ &- BR^{-1}B^\top P(T - (t \bmod \tau))\boldsymbol{\epsilon}(t) \\ &- f(\boldsymbol{h}^*_{(\psi^*,\phi^*)}(t), \tilde{\boldsymbol{u}}^*_T(t)) + A\boldsymbol{h}^*_{(\psi^*,\phi^*)}(t) \\ &+ B\tilde{\boldsymbol{u}}^*_T(t) - \boldsymbol{w}(t). \end{aligned} \tag{29}$$

Using the variation of constants formula again and Gronwall lemma [55] gives:

$$\begin{aligned}||\boldsymbol{\epsilon}(t)|| \leq e^{K(L+1)\tau}(KL||\boldsymbol{h}^*_{(\psi^*,\phi^*)}(t)||_{l^1(0,t)} \\ + \tau M_\infty ||\boldsymbol{w}||_{L^\infty(0,t)}).\end{aligned} \tag{30}$$

Combing Equation (28) and (30) finally yields

$$\begin{aligned}||\boldsymbol{h}_{(\psi^*,\phi^*)}(t)|| \leq M_\infty e^{-\mu t}||\boldsymbol{h}^*_{(\psi^*,\phi^*)}(t_1)|| \\ + \frac{1-e^{-\mu t}}{\mu}K(1+(L+1)\tau e^{K(L+1)\tau})||\boldsymbol{w}||_{l^\infty(0,t)},\end{aligned} \tag{31}$$

where $\mu = \mu_\infty - K_1 e^{-2\mu_\infty(T-\tau)} - K_2 L - KL(L+1)\tau e^{K(L+1)\tau}$. □

C Theorem 2 and Proof

Theorem (Generalizability). *Consider T, τ, K_2, μ_∞, and $\boldsymbol{h}_{(\psi^*,\phi^*)}(t)$ defined in Theorem 1 and let $\boldsymbol{u}_{(\psi^*,\phi^*)}(t)$ denote the optimal control action after NPC training in Algorithm 1 . Let $\boldsymbol{u}^*_\infty(t)$ denote the optimal solution of applying the linear model in Assumption 1 to the infinite-horizon minimization problem, defined in Equation (17) in Appendix B. Let $\boldsymbol{h}^*_\infty(t)$ denote the state controlled by $\boldsymbol{u}^*_\infty(t)$ using the nonlinear model $f_{\phi^*}(\cdot)$. There exists a constant K_3 such that:*

$$\begin{aligned}&||\boldsymbol{h}_{(\psi^*,\phi^*)}(t) - \boldsymbol{h}^*_\infty(t)|| + ||\boldsymbol{u}_{(\psi^*,\phi^*)}(t) - \boldsymbol{u}^*_\infty(t)|| \\ &\leq K_3 e^{-2\mu_\infty(T-\tau)}\Big(\frac{L+1}{\mu_\infty - K_2 L}||\boldsymbol{h}||_{l^1(0,t)} + ||\boldsymbol{h}(t)||\Big) \\ &+ K_3\tau e^{K_3(L+1)\tau}\frac{L+1}{\mu_\infty - K_2 L}(L||\boldsymbol{h}||_{l^1(0,t)} + ||\boldsymbol{w}||_{l^\infty(0,t)}),\end{aligned} \tag{32}$$

where $||\cdot||_{l^1(0,t)}$ is the l-1 norm on the function space over $(0,t)$.

Proof. Recall that the infinite-horizon cost minimization is in Equation (17) with the SSM model gives optimal state and control action trajectories in Equation (19). By definition, $\boldsymbol{u}^*_\infty(t) = \tilde{\boldsymbol{u}}^*_\infty(t) = R^{-1}B^\top P(T - (t \bmod \tau))\tilde{\boldsymbol{h}}^*_T(t)$ in Equation (19). Thus, we have:

$$\begin{aligned}\frac{d\boldsymbol{h}^*_\infty(t)}{dt} &= f(\boldsymbol{h}^*_\infty(t), \boldsymbol{u}^*_\infty(t)) + \boldsymbol{w}(t) + A_\infty \boldsymbol{h}^*_\infty(t) \\ &\quad - A\boldsymbol{h}^*_\infty(t) - B\boldsymbol{u}^*_\infty(t),\end{aligned} \tag{33}$$

where $A_\infty = A - BR^{-1}B^\top P_\infty$ is defined in Equation (19). Then, we evaluate the difference $\boldsymbol{e}(t) = \boldsymbol{h}_{(\psi^*,\phi^*)}(t) - \boldsymbol{h}^*_\infty(t)$ by Equations (33) and (26).

$$\begin{aligned}\frac{d\boldsymbol{e}(t)}{dt} &= A_\infty \boldsymbol{e}(t) + (A_{T,\tau}(t) - A_\infty)\boldsymbol{h}_{(\psi^*,\phi^*)}(t) \\ &\quad - BR^{-1}B^\top P(T - (t \bmod \tau))\boldsymbol{\epsilon}(t) \\ &\quad + f(\boldsymbol{h}_{(\psi^*,\phi^*)}(t), \tilde{\boldsymbol{u}}^*_T(t)) - f(\boldsymbol{h}^*_\infty(t), \boldsymbol{u}^*_\infty(t)) \\ &\quad - A\boldsymbol{e}(t) - B(\tilde{\boldsymbol{u}}^*_T(t) - \boldsymbol{u}^*_\infty(t)).\end{aligned} \tag{34}$$

By the last condition in Assumption 1,

$$\begin{aligned}&||f(\boldsymbol{h}_{(\psi^*,\phi^*)}(t), \tilde{\boldsymbol{u}}^*_T(t)) - f(\boldsymbol{h}^*_\infty(t), \boldsymbol{u}^*_\infty(t)) - A\boldsymbol{e}(t) \\ &- B(\tilde{\boldsymbol{u}}^*_T(t) - \boldsymbol{u}^*_\infty(t))|| \\ &\leq L(||\boldsymbol{e}(t)|| + ||\tilde{\boldsymbol{u}}^*_T(t) - \boldsymbol{u}^*_\infty(t)||) \\ &\leq L((1+K_2')||\boldsymbol{e}(t)|| + K_2'||\boldsymbol{\epsilon}(t)|| \\ &+ Ke^{-2\mu_\infty(T-\tau)}||\boldsymbol{h}_{(\psi^*,\phi^*)}(t)||),\end{aligned} \tag{35}$$

where $K_2' = ||R^{-1}B^\top||(K_0 + ||P_\infty||)$ and K_0 and P_∞ are defined in Equation (24). The last inequality holds because we have:

$$\begin{aligned}&\tilde{\boldsymbol{u}}^*_T(t) - \boldsymbol{u}^*_\infty(t) \\ &= -R^{-1}B^\top(P(T - (t \bmod \tau))\tilde{\boldsymbol{h}}^*_T(t) - P_\infty \boldsymbol{h}^*_\infty(t)) \\ &= -R^{-1}B^\top(P(T - (t \bmod \tau))\boldsymbol{\epsilon}(t) + \\ &\quad (P(T - (t \bmod \tau)) - P_\infty)\boldsymbol{h}_{(\psi^*,\phi^*)}(t) + P_\infty \boldsymbol{e}(t)).\end{aligned} \tag{36}$$

Then, we apply the variation of constant formula to Equation (34) and take norms. By Equation (35), we have:

$$\begin{aligned}||\boldsymbol{e}(t)|| &\leq K_2 L\int_0^t e^{-\mu_\infty(t-s)}||\boldsymbol{e}(s)||ds + \\ &\quad K(L+1)e^{-2\mu_\infty(T-\tau)}||\boldsymbol{h}_{(\psi^*,\phi^*)}(t)||_{l^1(0,t)} \\ &\quad + K(L+1)\int_0^t e^{-\mu_\infty(t-s)}||\boldsymbol{\epsilon}(s)||ds \\ &\leq K_2 L\int_0^t e^{-\mu_\infty(t-s)}||\boldsymbol{e}(s)||ds \\ &\quad + K(L+1)((e^{-2\mu_\infty(T-\tau)} + L\tau e^{K(L+1)\tau}) \\ &\quad ||\boldsymbol{h}_{(\psi^*,\phi^*)}(t)||_{l^1(0,t)} + \tau e^{K(L+1)\tau}||\boldsymbol{w}||_{l^\infty(0,t)}) \\ &\leq \frac{1}{\mu_\infty - K_2 L}K(L+1)((e^{-2\mu_\infty(T-\tau)} + L\tau e^{K(L+1)\tau}) \\ &\quad ||\boldsymbol{h}_{(\psi^*,\phi^*)}(t)||_{l^1(0,t)} + \tau e^{K(L+1)\tau}||\boldsymbol{w}||_{l^\infty(0,t)})\end{aligned} \tag{37}$$

where $K_2 = M_\infty(1+K_2')$ and the last second and the last inequalities is derived by Gronwall's lemma. Similarly, we can take norm of Equation (36) and finally obtain:

$$\begin{aligned}&||\boldsymbol{h}_{(\psi^*,\phi^*)}(t) - \boldsymbol{h}^*_\infty(t)|| + ||\boldsymbol{u}_{(\psi^*,\phi^*)}(t) - \boldsymbol{u}^*_\infty(t)|| \\ &\leq K_3 e^{-2\mu_\infty(T-\tau)}\Big(\frac{L+1}{\mu_\infty - K_2 L}||\boldsymbol{h}||_{l^1(0,t)} + ||\boldsymbol{h}(t)||\Big) \\ &+ K_3\tau e^{K_3(L+1)\tau}\frac{L+1}{\mu_\infty - K_2 L}(L||\boldsymbol{h}||_{l^1(0,t)} + ||\boldsymbol{w}||_{l^\infty(0,t)}),\end{aligned} \tag{38}$$

where $\boldsymbol{u}_{(\psi^*,\phi^*)}(t) \approx \tilde{\boldsymbol{u}}^*_T(t)$ by the second condition. □

D Appendix C: Synthetic Data Generation

For training dataset, it has 50 samples per class, each with 100 time steps. Mathematically,

$$\begin{cases} y^{class_0} = 7 + \sin(t) + \cos(t) + 0.2 \cdot N, \\ y^{class_1} = 2\sin(t) + 2\cos(t) + 0.2 \cdot N \end{cases} \tag{39}$$

where N is the random noise sampled from a normal distribution $\mathcal{N}(0,1)$. The generated data for both classes is combined, reshaped to include a singleton dimension, and labeled, creating an array of shapes of $(100, 100, 1)$ for the time series data and a corresponding label array.

For the test dataset, we consider a more complex scenario. Class 0 has 20 different types of time series patterns, each repeated 50 times, resulting in a total of 1000 samples. The time steps are generated as 100 evenly spaced values between 0 and 6. To introduce variability, for each of the 20 Class 0 types, a parabolic abnormal noise effect is calculated and added to the base sine and cosine waveform. The data for both classes is combined into a single dataset and reshaped to include a singleton dimension, resulting in an array of shapes (1050, 100, 1). Labels are created as an array of 1000 zeros for Class 0 and 50 ones for Class 1.